**LOW-LEVEL DESIGN**

**RAG-Based Customer Support Assistant**

**IMPLEMENTATION INTERNALS & DETAILED DESIGN**

## 1. Module-Level Design

The system is composed of modular components implemented using Python and LangChain-based libraries. Each module performs a specific function in the RAG pipeline.

| Module | Responsibility | Libraries |
|---|---|---|
| DocumentProcessingModule | Reads PDF and extracts text | PyPDFLoader |
| ChunkingModule | Splits text into chunks (300 size, 30 overlap) | RecursiveCharacterTextSplitter |
| EmbeddingModule | Converts chunks into embeddings | Sentence Transformers |
| VectorStorageModule | Stores embeddings in vector DB | ChromaDB |
| RetrievalModule | Retrieves top-k relevant chunks | ChromaDB Retriever |
| QueryProcessingModule | Builds prompt and calls LLM | Ollama |
| GraphExecutionModule | Executes LangGraph workflow | LangGraph |
| HITLModule | Handles escalation via CLI | Python input |

---

## 2. Data Structures

### 2.1 Core Types

**Chunk**

- text: str
- metadata: dict

**Embedding**

- embedding: List[float]

**Answer**

- query: str
- response_text: str
- decision: str ("ANSWER" / "ESCALATE")

**GraphState (TypedDict)**

class State(TypedDict):

query: str

answer: str

decision: str

### 2.2 Query–Response Schema

```
{
  "query": "What is refund policy?",
  "response": {
    "answer": "Refund is available within 7 days...",
    "decision": "ANSWER"
  }
}
```

---

### 3. LangGraph Workflow Design

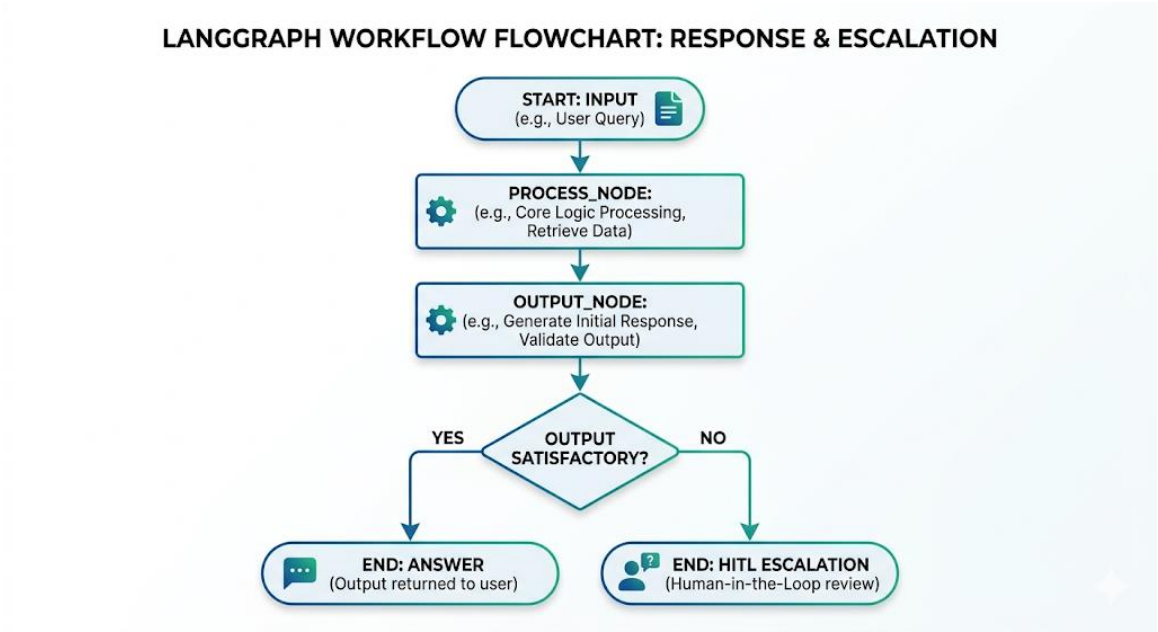
The system uses LangGraph to manage execution flow.

**Nodes:**

- process_node → handles retrieval + LLM
- output_node → handles decision + output

**Workflow:**

START → process_node → output_node → END



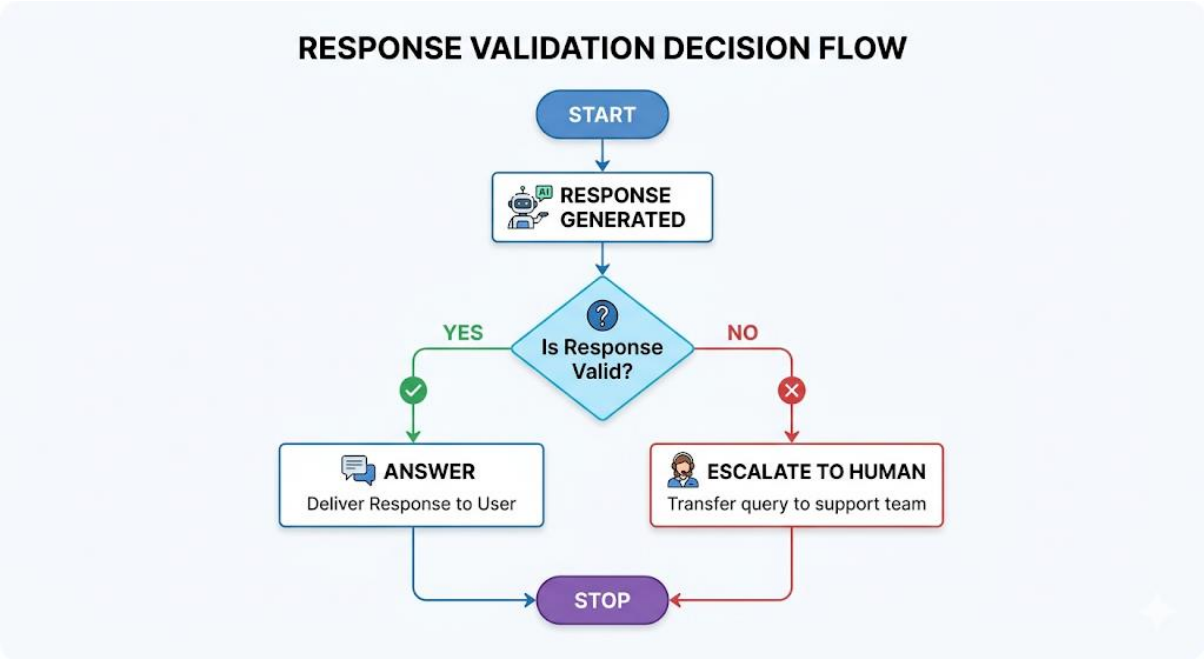

**Node Responsibilities**

| Node | Function | Input | Output |
|---|---|---|---|
| process_node | Retrieves + generates answer | query | answer, decision |
| output_node | Outputs or escalates | decision | final response |

---

## 4. Conditional Routing Logic

The system uses rule-based decision logic.



**Escalation Conditions:**

- No relevant chunks retrieved
- LLM returns “NOT_FOUND”
- Response contains phrases like:
  - "not available"
  - "no information"
- Response length is too short

**Example Logic:**

```
if "not_found" in response or len(response) < 40:
    decision = "ESCALATE"
else:
    decision = "ANSWER"
```

---

**5. HITL Design**

**Trigger**

When decision = ESCALATE

**Flow**

1. System detects low confidence
2. Prompts user for human input
3. Human enters response via CLI
4. Final response returned

---

**Implementation Example:**

```
print("Escalating to human...")
human = input("Human Agent: ")
```

**6. Interface Design**

**CLI Interface**

Input:

You: What is refund policy?

Output:

Bot: Refund is available within 7 days...

---

**7. Error Handling**

| Error | Handling |
|---|---|
| No chunks found | Escalate |
| Empty response | Escalate |
| LLM slow | Retry |
| Invalid query | Ask user again |